



PRÉSENTATION TECHNIQUE

Les conditions et les boucles en PHP

Faire des choix et répéter des actions avec PHP

Guide simple et détaillé — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Les conditions : faire des choix

DÉFINITION

Une condition est une instruction qui permet à PHP de **tester si quelque chose est vrai ou faux**, et d'exécuter un bloc de code différent selon le résultat. C'est ce qui donne à un programme la capacité de "réfléchir".

ANALOGIE SIMPLE

C'est exactement comme dans la vraie vie : "S'il pleut, je prends un parapluie. Sinon, je sors sans rien." PHP fonctionne pareil : il vérifie une condition (il pleut ?) et choisit quel bloc de code exécuter en fonction de la réponse.

La structure if (si)

La structure la plus simple est `if`, qui exécute un bloc de code seulement si la condition entre parenthèses est vraie.

```
<?php
    $age = 20;

    if ($age >= 18) {
        echo "Tu es majeur.";
    }

?>
```

La structure if / else (si / sinon)

On ajoute `else` pour exécuter un autre bloc de code quand la condition est fausse.

```
<?php
    $age = 15;

    if ($age >= 18) {
        echo "Tu es majeur.";
    } else {
        echo "Tu es mineur.";
    }

?>
// Résultat : Tu es mineur.
```

La structure if / elseif / else (plusieurs choix)

Quand il y a plus de deux possibilités, on utilise `elseif` pour ajouter des conditions intermédiaires.

```
<?php
    $note = 14;

    if ($note >= 16) {
        echo "Très bien";
    } elseif ($note >= 12) {
        echo "Bien";
    } elseif ($note >= 10) {
        echo "Passable";
    } else {
        echo "Insuffisant";
    }

?>
// Résultat : Bien
```

ASTUCE PÉDAGOGIQUE

Pour expliquer à tes camarades : PHP teste les conditions **dans l'ordre, une par une**, et s'arrête dès qu'il en trouve une vraie. Il n'exécute jamais deux blocs à la fois, même si plusieurs conditions sont vraies.

2. Les opérateurs utilisés dans les conditions

Les opérateurs de comparaison

Ce sont les outils qui permettent de comparer deux valeurs à l'intérieur d'une condition.

Opérateur	Signification	Exemple
<code>==</code>	Égal à (valeur)	<code>\$a == \$b</code>
<code>===</code>	Strictement égal (valeur ET type)	<code>\$a === \$b</code>
<code>!=</code>	Différent de	<code>\$a != \$b</code>
<code>></code>	Supérieur à	<code>\$a > \$b</code>
<code><</code>	Inférieur à	<code>\$a < \$b</code>
<code>>=</code>	Supérieur ou égal à	<code>\$a >= \$b</code>
<code><=</code>	Inférieur ou égal à	<code>\$a <= \$b</code>

PIÈGE FRÉQUENT : `==` VS `===`

Avec `==`, PHP compare seulement les **valeurs** et peut convertir les types automatiquement : `"5" == 5` donne `true`. Avec `===`, PHP compare aussi le **type** : `"5" === 5` donne `false` car l'un est un texte et l'autre un nombre. Il est recommandé d'utiliser `===` pour éviter les surprises.

Les opérateurs logiques

Ils permettent de combiner plusieurs conditions ensemble.

Opérateur	Signification	Exemple
<code>&&</code>	ET (les deux doivent être vraies)	<code>\$age >= 18 && \$a_permis</code>
<code> </code>	OU (au moins une doit être vraie)	<code>\$samedi \$dimanche</code>
<code>!</code>	NON (inverse le résultat)	<code>!\$estConnecte</code>

```
<?php
    $age = 20;
    $aPermis = true;

    if ($age >= 18 && $aPermis) {
        echo "Tu peux conduire.";
    } else {
        echo "Tu ne peux pas conduire.";
    }

?>
// Résultat : Tu peux conduire.
```

ANALOGIE SIMPLE

`&&` (ET), c'est comme dire "il faut avoir 18 ans ET avoir le permis" : les deux conditions sont obligatoires. `||` (OU), c'est comme dire "on sort samedi OU dimanche" : une seule des deux suffit pour que ce soit vrai.

3. Les boucles : répéter des actions

DÉFINITION

Une boucle permet d'exécuter **le même bloc de code plusieurs fois**, sans avoir à le recopier manuellement. C'est essentiel pour éviter de répéter du code inutilement.

La boucle for

On utilise `for` quand on connaît à l'avance le **nombre exact de répétitions**.

```
<?php
    for ($i = 1; $i <= 5; $i++) {
        echo "Tour numéro " . $i . "<br>";
    }
?>
// Résultat : Tour numéro 1, Tour numéro 2 ... jusqu'à Tour numéro 5
```

La boucle `for` contient toujours 3 parties séparées par des points-virgules :

- 1 `$i = 1` : le point de départ (initialisation), exécuté une seule fois.
- 2 `$i <= 5` : la condition vérifiée avant chaque tour. Tant qu'elle est vraie, la boucle continue.
- 3 `$i++` : ce qui se passe à la fin de chaque tour (ici, on ajoute 1).

La boucle while

On utilise `while` ("tant que") quand on ne connaît **pas forcément à l'avance** le nombre de répétitions, mais qu'on connaît la condition d'arrêt.

```
<?php
    $compteur = 0;

    while ($compteur < 3) {
        echo "Compteur : " . $compteur . "<br>";
        $compteur++;
    }
?>
// Résultat : Compteur : 0, Compteur : 1, Compteur : 2
```

PIÈGE FRÉQUENT : LA BOUCLE INFINIE

Si on oublie d'écrire `$compteur++`, dans une boucle `while`, la condition reste vraie pour toujours et le programme tourne sans jamais s'arrêter. Il faut toujours vérifier que la condition finira par devenir fausse.

La boucle do...while

Variante de `while`, mais qui exécute le bloc de code **au moins une fois**, même si la condition est fausse dès le départ, car la condition est vérifiée à la fin.

```
<?php
    $i = 10;
    do {
        echo "Valeur : " . $i;
        $i++;
    } while ($i < 5);
?>
// Résultat : Valeur : 10 (affiché une fois, même si la condition est déjà fausse)
```

4. La boucle foreach et le contrôle des boucles

La boucle foreach

La boucle `foreach` est spécialement conçue pour **parcourir un tableau** (array), élément par élément, sans avoir besoin de gérer un compteur manuellement.

```
<?php
$fruits = ["mangue", "banane", "orange"];

foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}

?>
// Résultat : mangue, puis banane, puis orange
```

Pour un tableau associatif, `foreach` peut aussi récupérer la clé et la valeur en même temps :

```
<?php
$etudiant = [
    "prenom" => "Khadija",
    "ville" => "Dakar"
];

foreach ($etudiant as $cle => $valeur) {
    echo $cle . " : " . $valeur . "<br>";
}

?>
// Résultat : prenom : Khadija, puis ville : Dakar
```

ANALOGIE SIMPLE

`foreach`, c'est comme distribuer des copies à toute une classe : tu prends chaque élève un par un, dans l'ordre, jusqu'à ce qu'il n'y en ait plus. Tu n'as pas besoin de compter les élèves toi-même, PHP s'en charge.

Sortir d'une boucle avec break

L'instruction `break` arrête complètement la boucle, même si la condition était encore vraie.

```
<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i == 4) {
        break; // arrête complètement la boucle
    }
    echo $i;
}

?>
// Résultat : 1 2 3 (s'arrête avant d'atteindre 4)
```

Passer un tour avec continue

L'instruction `continue` n'arrête pas la boucle, elle saute simplement le tour en cours et passe directement au suivant.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    if ($i == 3) {
        continue; // saute uniquement le tour numéro 3
    }
    echo $i;
}

?>
// Résultat : 1 2 4 5 (le 3 est sauté, mais la boucle continue)
```

5. Résumé pour bien expliquer conditions et boucles

Quand utiliser quelle structure ?

Structure	Utilisation
<code>if / else</code>	Faire un choix simple entre deux possibilités
<code>if / elseif / else</code>	Faire un choix entre plusieurs possibilités
<code>for</code>	Répéter un nombre de fois connu à l'avance
<code>while</code>	Répéter tant qu'une condition reste vraie
<code>do...while</code>	Répéter au moins une fois, puis vérifier la condition
<code>foreach</code>	Parcourir tous les éléments d'un tableau
<code>break</code>	Arrêter complètement une boucle
<code>continue</code>	Sauter le tour en cours et continuer la boucle

La phrase à retenir pour ta présentation

"Les conditions (if/else) permettent à PHP de faire un choix selon une situation, comme un humain. Les boucles (for, while, foreach) permettent de répéter une action plusieurs fois sans réécrire le code, ce qui rend les programmes plus courts et plus puissants."

Erreurs courantes à éviter

- Confondre `=` (affectation) et `==` (comparaison) dans une condition.
- Oublier d'incrémenter la variable dans une boucle `while`, ce qui crée une boucle infinie.
- Utiliser `for` alors qu'on ne connaît pas le nombre de répétitions à l'avance (préférer `while`).
- Oublier les accolades `{ }` autour du bloc de code, surtout avec plusieurs lignes.
- Confondre `break` (arrête tout) et `continue` (saute juste un tour).